Quantum Genome Detective · consultation notes

Start with the notebook. These notes are for selective consultation, not a prerequisite reading assignment. Allow approximately **2–3 hours** for the guided foundation, excluding installation, QPU queues and open investigation. This is an estimate, not participant-tested timing.

You need this for the core lab

The challenge in one page

Imagine knowing that two genetic variants are present, but not whether they occur on the same chromosome copy. Haplotype phasing reconstructs which alleles occur together. This can matter for interpreting variants in a gene: two alternate alleles in **cis** are on one copy; in **trans** they are on opposite copies. The teaching example makes no clinical prediction.

Humans normally carry two copies of each autosomal chromosome. The lab restricts attention to known, heterozygous, biallelic SNP positions. Each site therefore has one 0 allele and one 1 allele across the two copies. A candidate bit vector `x` gives one copy; `1-x` gives the other. Swapping these copy names changes no relationship.

Sequencing provides incomplete fragments from individual copies, with the origin labels removed. Some observations can be wrong. The lab asks whether accumulated relationships between sites can help reconstruct a complementary pair, then explores how a quantum circuit samples possible answers.

The route is **aligned reads → evidence → signed graph → pairwise cost → exact enumeration → Ising → QAOA → compilation/noise**. A signed graph represents preferences, not certainty. An exact minimum certifies the supplied mathematical model, not the true biological reconstruction. No quantum advantage is assumed.

Your exercises build evidence, score candidates, enumerate the tiny search space, encode Ising energy, construct complete QAOA layers, calculate expected energy and choose a comparison. Supplied helpers handle pictures, optimization, measurement decoding and hardware transport. Use the final unblocking appendix after an attempt and disclose the definitions you used.

Simplified data assumptions

Rows are reads; columns are **already aligned to known SNP positions**. Entries are `0`, `1` or `-1` for missing. Alleles are labels specific to each site, not literal DNA bases. This is not variant calling, raw-read alignment or genome assembly. The tutorial uses five SNPs and a fixed embedded matrix; the pictured synthetic pair supplies intuition and is never an optimizer input.

A row contributes one vote for every pair of observed sites, even non-adjacent sites. A row `0,1,1` contributes opposite votes on 1–2 and 1–3, and a same vote on 2–3. Its complement contributes identical relationships. Missing calls contribute nothing. Repeated rows count repeatedly in this starter; deduplication and confidence weighting are possible investigations.

A read covering L sites generates L(L−1)/2 pair votes. These are correlated: a single error can affect several edges. More votes do not automatically mean more independent evidence. There are no read qualities or learned error probabilities in the baseline.

Graph cost and Ising mapping

Store each pair once in the upper triangle. Let A count same votes and D count opposite votes. A candidate pays A when its bits differ, D when they agree:

$$C(x) = \sum_{i<j} \left(A_{ij}[x_i \neq x_j] + D_{ij}[x_i = x_j] \right).$$

The graph's net weight A−D prefers same when positive and opposite when negative. Edge width shows the absolute net strength; labels show both counts. A tie gives no active coupling but still contributes a constant cost. A graph drawn with no edge does not distinguish a tie from no observations unless you inspect the counts.

With spins $z_i = 1 - 2x_i$,

$$J_{ij} = (D_{ij} - A_{ij})/2, \qquad c_0 = \tfrac{1}{2}\sum_{i<j}(A_{ij} + D_{ij}), \qquad E(x) = c_0 + \sum_{i<j} J_{ij} z_i z_j.$$

Negative J favors equal bits; positive J favors opposite bits. Three same votes and one opposite vote give energy $2-z_iz_j$: 1 for equal bits and 3 for different bits. Keep the offset for reported energies. It only produces a global phase in the cost circuit, so gates can omit it.

The notebook exhaustively verifies `E(x)=C(x)` and complement symmetry on all 32 states. Enumeration returns every state with its cost and identifies all minimum labels. This is the core classical reference and a useful debugging tool. Its exponential growth prevents treating it as a scalable solution.

QAOA baseline

In R2P you used QAOAAnsatz for MaxCut. Here you construct the layers yourself. Apply H to initialize every qubit in |+⟩. In each of p layers, apply the graph's cost operations followed by an X mixer. Finally measure a separate copy of the trained circuit.

For parameters gamma and beta, use `rzz(2*gamma*J[i,j]/scale)` and `rx(2*beta)`, with `scale=max(1,max(abs(J)))`. Qiskit rotation gates include a factor of one half in their exponent, which explains the factor two. Scaling changes the angle convention; optimization and reporting still use the original unscaled energies. There are no local fields here, so no RZ cost gates are needed.

The unmeasured ansatz supports exact statevector probabilities. Your expected energy is the sum of each probability times its basis state's energy. The supplied COBYLA loop tries two seeded starting points and selects the lower final expectation. It uses **no training shots**. Statevector simulation itself grows exponentially and is a teaching tool for this small case. Finite-shot optimization is an extension.

The measured copy generates 2,048 fresh samples after training. Qiskit's printed key has qubit 0 on the **right**; notebook candidate text has SNP 1 on the **left**. Use the supplied decoder once. Statevector index k has bit i equal to `(k >> i) & 1`; do not assume enumeration rows use that order.

Choose a meaningful comparison: mean cost describes typical sample quality, while optimum frequency measures repeated minimum finding. The display includes the exact minimum, the best sampled candidate, and the full sampled cost distribution. Uniform probabilities are a useful null comparison. Finding one optimum is weaker evidence than concentrating sampling mass there. One seed and one tiny instance do not establish general usefulness or speed advantage.

Short IBM hardware guidance

First train locally. Then transpile the fixed measured circuit for FakeManilaV2 and simulate its snapshot noise. Inspect depth and two-qubit gate count: routing can increase both. FakeManilaV2 is a historical five-qubit device snapshot, not a current calibration. Aer approximates device noise; it does not reproduce every hardware effect.

Transpilation chooses physical layout, inserts routing and decomposes gates. It does not retrain variational parameters. Compare ideal and noisy distributions using the same observations, objective, parameters and shots. The `meas` classical register retains logical measurement order after routing; reversing the physical layout again corrupts decoding.

Complete the fake baseline first, then run the same fixed circuit and trained parameters on IBM if you have access. Real IBM access is optional and encouraged for a focused study. The optional hardware section appears immediately after the fake comparison and before the open investigation. It uses an in-memory Runtime connection, authorized backend selection, preset pass-manager compilation and `SamplerV2(mode=backend)`. It requests a key through a hidden prompt and never saves an account. **Never paste tokens into notebook cells or outputs.** Backend name/instance configuration depends on your allocation.

If a real result is obtained, the same compact table compares ideal simulation, FakeManila snapshot noise and real IBM hardware using mean cost, optimum frequency, depth and shots. With the run disabled, no real-hardware row appears. QPU queue time is outside the guided-lab estimate. Save backend, shots, job ID, reads and trained parameters before waiting for a result. On reconnect, retrieve the existing job with `service.job(saved_job_id)` rather than resubmit it. Record queue failures or unavailable access as limitations. Hosted Colab installation/UI, live authorization, queues and real submission remain pending organizer validation; local fake-backend success does not verify them.

Read this if you choose this investigation

Choose one focused question. State one proposed change, one fair comparison and one conclusion, limitation or failure. Keep a baseline copy and change one factor at a time when you want a causal interpretation. Record relevant seeds/settings; repeat where the claim needs it. Recompute exact scores after changing evidence or the objective.

Evidence and objective

Does long-read pair expansion overcount confidence? Compare equal pair votes with normalized per-read weights. Do suspicious relationships help or hurt? Prune weak edges and track both reconstruction scores and circuit resources. If you introduce a new metric, evaluate both baseline and modified method under the same stated criteria.

An optional alternative, full-read Minimum Error Correction, counts the fewest observed allele calls to change to fit either complementary copy. For a read with L observed calls and d mismatches to x, its contribution is `min(d,L−d)`. Pairwise cost instead contributes `d*(L−d)` for that read. They can rank candidates differently. Neither objective guarantees biological truth; evaluate against a separately known synthetic truth only when your data generation supports that interpretation.

QAOA and search

Study depth, optimizer, initialization, mixer choices or finite-shot training. Report optimization effort as well as final shots. If you select the best of several restarts, disclose that selection. Deeper circuits may help ideal search yet degrade under noise. A heuristic classical baseline can extend the exact reference, but it is not another mandatory implementation.

Hardware and resources

Study coupling-aware SNP ordering, initial layout, transpiler seeds, routing or evidence pruning. A lower two-qubit gate count may reduce noise without improving ideal performance. Hold trained angles fixed to isolate compilation effects. If you retrain at each depth or layout, explain what changed and what stayed controlled. Hardware-in-the-loop optimization and mitigation are optional studies, not prerequisites.

Information limits

Remove bridging reads and inspect the graph's connected components. With k disconnected active components, independently flipping each component leaves the pairwise score unchanged; modulo the global copy swap this gives at least 2^(k−1) equivalent relative orientations. More search cannot recover a relationship absent from the model. Tied net evidence may also remove information, and connected graphs can remain ambiguous because of contradictions or symmetries.

Judging and minimal submission

Submit a notebook/code with outputs and a short presentation. Show the question, proposed change, baseline/comparison, evidence and conclusion/limitation. Identify what your team implemented, which fallbacks or external references you used, and your original contribution. Hardware claims require backend, shots and job IDs; no hardware run is required to complete the lab.

Judging relies on presentations and submitted evidence with manual/AI-assisted review. There is no evaluation server or hidden leaderboard. Running the supplied workflow is a learning achievement; competitive work investigates beyond it. A well-supported negative result or failure analysis can be a contribution. More QPU jobs or a single flattering number do not by themselves establish scientific value.

Small reference shelf

- Browning & Browning, *Haplotype phasing: existing methods and new developments*: scientific background beyond the simplified model.
- Farhi, Goldstone & Gutmann, *A Quantum Approximate Optimization Algorithm*: original QAOA formulation.
- IBM: bit ordering: reconcile candidate, register and statevector conventions.
- IBM: local testing mode: what fake backends do and do not establish.
- IBM: SamplerV2 API and Sampler results: optional hardware section interfaces.